Testing the Preconditions for Cross-Database Generalisation of Machine Learning-Driven Replication Mode Selection: A Controlled Synthetic Study Across PostgreSQL, MySQL, and MongoDB

Imane Jerrari[1*] and Ismail Assayad[1]

[1*]Laboratory of Information Systems (LIS), Faculty of Sciences Ain Chock & ENSEM, Hassan II University of Casablanca, Casablanca, 20000, Morocco.

*Corresponding author(s). E-mail(s): imane.jerrari-etu@etu.univh2c.ma;
Contributing authors: i.assayad@ensem.ac.ma;

Abstract

Our prior work introduced IAHA, a hybrid machine-learning framework selecting among four replication modes (PERFORMANCE, CONSISTENCY, AVAILABILITY, ECONOMIC) for containerised PostgreSQL, leaving generalisation to other engines as future work. Here, under a controlled design where all three offline datasets share a single synthetic generator, we test whether a per-engine RandomForest classifier behaves consistently across PostgreSQL, MySQL, and MongoDB, and we validate live Kubernetes failover on all three engines. Independent classifiers (PostgreSQL, $n = 644$; MySQL, $n = 200$; MongoDB, $n = 200$) significantly outperform a static CPU-threshold heuristic by **+18.0** to **+21.0** percentage points (McNemar $p < 10^{-6}$), with mean accuracy **95.90%–98.62%**; a direct comparison shows RandomForest matches the strongest alternative (XGBoost) and both outperform linear/kernel classifiers. A size-matched control shows part of PostgreSQL's advantage comes from dataset size (a **1.93**-point drop at $n = 200$), but size-matched PostgreSQL nominally lands between, not below, MySQL and MongoDB (exploratory; borderline or non-significant after correction), so size alone does not fully explain cross-engine differences. Feature-importance rankings correlate strongly across engines (Spearman $\rho = 0.81$–0.92). All three classifiers misclassify AVAILABILITY as CONSISTENCY; given small per-engine counts ($n = 1, 3, 5$), we treat this descriptively; SMOTE improves recall only to **20%**. An exploratory live failover comparison finds total recovery time differs between deployments (PostgreSQL–MySQL and MySQL–MongoDB significant after Bonferroni correction; PostgreSQL–MongoDB not), confounded by an orchestration asymmetry. Because the offline datasets share one generative process, those results establish consistency under controlled conditions, not generalisation to heterogeneous production systems.

Keywords: High Availability, Containerised Databases, Cross-Database Generalisation, Machine Learning, Replication Mode Selection

1 Introduction

1.1 Context and Motivation

Adaptive replication-mode selection has recently been shown to outperform static heuristics on containerised PostgreSQL clusters [1], using a RandomForest classifier trained on operational metrics to choose among four modes at runtime. That study, however, was scoped explicitly to a single database engine, and identified transfer learning to MySQL and MongoDB as a direct item of future work (FW4 in [1]). IAHA operates in two layers: a fast, rule-based P1 layer that applies static CPU-utilisation thresholds as an immediate fallback, and a P2 layer in which a RandomForest classifier is invoked at runtime on a ten-feature operational snapshot (Table 1) to select among the four replication modes. The

P2 classifier is trained offline on expert-validated operational traces and queried online at a fixed polling interval, with mode transitions applied through CloudNativePG's replication configuration; the static heuristic used as this paper's own baseline (Section 3.5) mirrors IAHA's P1 layer specifically, so that both this paper's comparison and IAHA's own evaluation isolate the value the P2 classifier adds over the simpler fallback it was designed to replace.

This leaves an open empirical question that matters beyond PostgreSQL: is the relationship between operational metrics (CPU utilisation, transaction throughput, replication lag, etc.) and the appropriate replication mode a property of PostgreSQL specifically, or a more general property of replicated database systems under load? If the latter, a single engine-agnostic feature schema could support adaptive high availability (HA) across heterogeneous database fleets — a common scenario in practice, where organisations operate relational and NoSQL engines side by side. As a first step toward answering that question, this paper asks a narrower, more tractable one: under controlled conditions that hold the data-generating process fixed across engines, does classifier behaviour at least remain consistent? A negative answer would undercut the premise of cross-engine transfer before any live multi-engine validation is attempted; a positive answer, while not proof of generalisation, would justify pursuing it.

1.2 Problem Statement

Under a controlled synthetic design that holds the data-generating process fixed across engines, does a machine-learning classifier for replication mode selection, trained independently on PostgreSQL, MySQL, and MongoDB using an identical engine-agnostic feature schema, achieve consistent classification performance across engines, and do the same classes remain difficult to predict regardless of the underlying engine?

1.3 Research Questions

RQ1 — Under a controlled synthetic design sharing a single generative process, is classifier performance (accuracy, macro-F1) consistent across PostgreSQL, MySQL, and MongoDB, or does it depend on engine-specific factors such as dataset size and class distribution?

RQ1b — Is the cross-engine difference in accuracy and PERFORMANCE-class recall explained by dataset size rather than by engine identity? (We test this exploratorily by subsampling PostgreSQL to the size of the MySQL and MongoDB datasets, $n = 200$.)

RQ1c — Is RandomForest, inherited from [1], actually competitive with alternative classifiers on this specific four-class mode-selection task, or would another model perform better?

RQ2 — Is the relative importance of operational features for mode selection stable across a relational engine (PostgreSQL), a second relational engine (MySQL), and a document-oriented engine (MongoDB)?

RQ3 — Does the AVAILABILITY-class failure mode identified as a limitation in [1] (L3: extreme class imbalance) show the same error pattern across engines, and is it consistent with a correctable sample-size artefact or with a structural feature-space overlap with CONSISTENCY? (Given the very small per-engine AVAILABILITY counts, we treat this as an exploratory, hypothesis-generating question rather than one this study is powered to settle definitively.)

RQ4 — Beyond offline classification accuracy, do live Kubernetes failover tests show consistent end-to-end recovery behaviour across the three engines, and what confounds (deployment orchestration, measurement artefacts, temporal effects) must be accounted for in interpreting any differences observed?

1.4 Contributions

- A controlled synthetic evaluation of replication-mode classification across three database engines (PostgreSQL, MySQL, MongoDB), generated by a single shared engine-agnostic generator and feature schema, isolating whether classification behaviour depends on engine identity. We are explicit that this design tests a precondition for cross-engine generalisation — consistency of classifier behaviour under a shared generative process — rather than generalisation to heterogeneous production systems itself (Section 3.2).

- A sample-size control experiment that directly disentangles engine identity from training-set size, showing that the apparent performance advantage of the largest dataset (PostgreSQL) is *partly* — but not fully — attributable to its size: size-matched PostgreSQL nominally lands between MySQL and MongoDB rather than below both (an exploratory ordering), so a residual, unattributed difference between engines may remain and is flagged as an open question rather than resolved.
- Statistically grounded evidence (BCa bootstrap confidence intervals, McNemar tests against a static heuristic baseline) that ML-driven mode selection significantly outperforms static thresholds on all three engines, not only PostgreSQL: a +18.0 to +21.0 percentage-point accuracy improvement, significant at $p < 10^{-6}$ on every engine (Section 4.1). This is the most unambiguous result in the paper and holds regardless of how the other, more exploratory findings below are read.
- A direct comparison of RandomForest against XGBoost, logistic regression, and RBF-SVM on the same task (Section 4.3), showing RandomForest is empirically competitive with the strongest alternative (XGBoost) rather than an unexamined default, and that both tree-based methods substantially outperform linear/kernel methods — a result requiring no hedging.
- Evidence of cross-engine stability in feature importance rankings (Spearman $\rho \geq 0.80$), motivating future transfer-learning work.
- A live Kubernetes failover validation across all three engines (Section 4.7), extending the offline classification results with measured end-to-end recovery times. We report this alongside, rather than instead of, the limitations we uncovered in the process: an orchestration asymmetry between engines, a temporal ordering effect in the PostgreSQL tests, and an unresolved measurement artefact in a subset of the MongoDB tests — so that the live results are read as an initial validation rather than a fully controlled comparison.

2 Related Work

This section positions the study against four strands of prior work: adaptive high availability for databases (Section 2.1), machine learning for database management (Section 2.2), explainable AI in critical systems (Section 2.3), and cross-database and transfer-learning approaches (Section 2.4). It closes by stating the research gaps this paper addresses (Section 2.5).

2.1 Adaptive High Availability for Databases

Existing PostgreSQL HA tools follow a common pattern: a single synchronous or asynchronous replication policy chosen once at deployment time and left unchanged at runtime. Patroni [2] and repmgr [3] provide consensus-based leader election but do not adapt replication behaviour to load; Stolon [4] follows the same fixed-policy pattern. CloudNativePG [5], the CNCF-donated Kubernetes operator used as the substrate in [1], improves failover latency through a native Kubernetes control loop but likewise applies one static replication configuration for the cluster's entire lifetime. None of these tools offers more than two operational modes, and none adapts mode selection to observed workload conditions — the gap that IAHA [1] addresses for PostgreSQL, and that this paper extends to MySQL and MongoDB.

2.2 Machine Learning for Database Management

ML-driven database optimisation is an active area spanning several problem classes: cardinality estimation and query optimisation (SageDB [6], Neo [7], Bao [8]), automated configuration tuning (OtterTune [9], CDBTune [10]), and resource management (Autopilot [11], FIRM [12]). A recurring finding in this literature, most directly relevant to our RQ1, is that ML-driven tuning gains do *not* automatically transfer across systems or even across versions of the same system: OtterTune [9] explicitly maps unseen workloads to previously characterised ones because tuning effort on one DBMS does not reduce tuning effort on another, and LlamaTune [21] reports that the throughput improvement of its tuning approach over a baseline optimiser narrows substantially when ported from PostgreSQL v9.6 to v13.6 on certain workloads (e.g. from 20.8% to 3.6% on YCSB-B), despite both being the same engine. Multi-DBMS evaluations such as GPTuner [22] and UDO [23], which benchmark

across PostgreSQL and MySQL, similarly report engine-dependent variation in tuning effectiveness rather than uniform gains. This body of work motivates our explicit empirical test (RQ1, RQ1b) of whether classification performance is consistent across engines or is confounded by other factors — in our case, training-set size — rather than assuming consistency holds by default. None of these systems addresses replication-mode selection specifically, nor provides operator-facing explainability, which remains the gap IAHA [1] and this paper target.

2.3 Explainable AI in Critical Systems

Explainable AI (XAI) addresses the interpretability of ML-driven decisions [13] through post-hoc methods such as SHAP [14] and LIME [15], counterfactual explanations [16], and rule extraction [17]. XAI has been applied to high-stakes domains including healthcare [18], autonomous vehicles [19], and finance [20], but — as noted in [1] — its application to database HA management remains largely unexplored. This paper does not introduce new XAI components beyond those already proposed in [1]; instead, the feature importance analysis in Section 4.4 serves a complementary diagnostic role, testing whether the same decision signature used for explainability on PostgreSQL is structurally stable enough to remain meaningful on MySQL and MongoDB.

2.4 Cross-Database and Transfer Learning Approaches

Despite extensive work on ML for single-DBMS optimisation (Section 2.2), explicit cross-engine generalisation remains comparatively underexplored. Most multi-DBMS evaluations in the tuning literature (e.g. GPTuner [22], UDO [23]) train and evaluate *separate* models per engine to demonstrate applicability, rather than testing whether a model or feature representation *transfers* from one engine to another — the same design choice we make in this paper (independent per-engine classifiers, Section 3), motivated by the same practical consideration: heterogeneous internal metrics and configuration spaces across engines make naive model reuse non-trivial.

Industrial efforts to copy or retrain ML models across database instances exist as systems-engineering patterns, but typically address operational mechanics of model transfer (e.g. avoiding redundant retraining) rather than evaluating whether the *learned relationships* generalise across heterogeneous engines. To our knowledge, no prior work evaluates whether the operational signals driving a replication-mode mapping learned on one database engine are structurally consistent with those of a different engine family (relational to relational, or relational to document-oriented) — the specific gap this paper addresses empirically through RQ1–RQ3. We note that this paper tests the *preconditions* for transfer (consistent classification behaviour, stable feature-importance signatures) rather than transfer itself; directly training on one engine and testing on another is identified as future work (FW1).

Why the systems cited above are not empirical baselines here. Section 2.2 discusses OtterTune, CDBTune, GPTuner, UDO, and LlamaTune at length because they are the closest prior work on ML-driven database management, and we return to them here to state explicitly why none is used as an empirical comparison point in this paper's results. These systems perform configuration *tuning* — selecting numerical parameter values (buffer sizes, timeouts, memory allocations) to optimise a continuous performance objective — whereas this paper performs replication-mode *selection*, a four-class classification problem over discrete operational states. Patroni, repmgr, and CloudNativePG (Section 2.1) are HA orchestration tools that execute a fixed failover policy rather than choosing among multiple named replication modes at all. Running any of these systems on our task would not produce a comparable output: a tuning system would need to be given a four-class decision problem it was not designed to solve, and an orchestration tool would need a mode-selection layer bolted on that does not exist in its original design. An empirical comparison under these conditions would not isolate a meaningful difference in method — it would compare our classifier against our own re-implementation of a decision layer these systems do not have, which we do not consider informative. We therefore restrict empirical comparison in this paper to (i) the static heuristic baseline (Section 3.5), which performs the same classification task under

a simpler decision rule, and (ii) alternative ML classifiers on the same task (Section 4.3), rather than attempting a system-level comparison that would not be apples-to-apples.

2.5 Research Gaps Addressed by This Paper

Building on the four gaps identified in [1] (binary modes, static thresholds, no explainability, no temporal optimisation), we identify a fifth gap specific to cross-engine generalisation:

Gap 5 – Untested Cross-Engine Generalisation: Existing ML-driven database management systems are evaluated on a single engine (or, where multiple engines are used, evaluated independently without testing whether learned relationships transfer between them), leaving open whether reported performance gains reflect properties of the engine, the dataset, or the underlying operational-metric relationships themselves. This paper addresses Gap 5 through the size-matched control analysis in Section 4.2, which shows dataset size is part, but not all, of the explanation for the cross-engine differences observed: it does not fully close the gap, since size-matched PostgreSQL nominally lands between MySQL and MongoDB rather than below both (an exploratory ordering), leaving a possible residual difference between engines at matched sample size open (Section 4.2, FW4), and — as discussed in Section 3.2 — closing it fully will additionally require moving beyond a single shared synthetic generator to independently instrumented engines.

3 Methodology

This section describes the controlled design used to answer RQ1–RQ4: the three synthetic datasets (Section 3.1), the scope of the study (Section 3.2), the classification protocol and the alternative-classifier comparison (Sections 3.3 and 3.4), the statistical tests, including the comparison families and the multiple-comparison correction (Section 3.5), the sample-size control (Section 3.6), and the live multi-engine failover validation (Section 3.7).

3.1 Datasets

Three datasets were used, sharing an identical ten-feature schema (Table 1) and four-class label space (PERFORMANCE, CONSISTENCY, AVAILABILITY, ECONOMIC). All three are produced by a *single shared synthetic generator* (a common `generate_samples(n, system)` Python function, confirmed by direct inspection of the generation script), called three times with the same random seed and the same scenario-based labelling logic, varying only the requested sample size n and an engine-specific parameter `system`. Concretely, each sample is drawn from one of five latent operating scenarios (normal, peak, off-peak, crisis, degraded), and the four-class mode label is then assigned by a deterministic rule cascade over the sampled feature values (load thresholds, error rates, deadlock counts, and time-of-day); this is what produces the class distributions in Table 2 below, and the full mechanical detail (scenario probabilities, parameter ranges, and the exact rule cascade) is given in Limitation L1 (Section 5.1) rather than repeated here. We state this design choice here, rather than only in Limitation L1, because it directly shapes how the cross-engine comparisons in Section 4 should be read: the three datasets are controlled parametric variations of one generative process, not independent instrumentation of three production systems. This guarantees a consistent, comparable feature schema by construction, but it also means the study should be understood as a controlled synthetic evaluation, not an empirical survey of real production traces (see Limitation L1 for the full generator detail and Threats to Validity for the consequences).

3.2 Scope: What This Study Does and Does Not Show

Because this design choice is central to interpreting every result that follows, we state it plainly before presenting the feature schema and class distributions. This study shows that a RandomForest classifier, trained independently per engine, behaves *consistently* — in accuracy, in error pattern, and in which features it relies on — across three parametric variations of a single synthetic generator that differ only in requested sample

size and a small number of engine-specific parameter ranges (Section 3.1; Limitation L1). This consistency is a *necessary precondition* for cross-database transfer learning to be worth pursuing: if a shared generative process produced inconsistent classifier behaviour across engines, there would be little reason to expect a model trained on one real engine to say anything useful about another.

This study does *not* show that a classifier trained on PostgreSQL data generalises to independently instrumented, live MySQL or MongoDB deployments. The three datasets do not capture engine-specific implementation details, workload idiosyncrasies, or telemetry artefacts that would appear in production traces collected separately from each engine; two features (`wal_buffers_full`, `trans_rolled_back`) vary in scale between engines within the generator, but the remaining eight, and the entire labelling-rule cascade, are identical by construction. Consequently, results such as the high cross-engine Spearman correlation in feature importance (Section 4.4) should be read as evidence that the *signature* of a good classifier is stable under this controlled design, not as evidence that the signature would remain stable if measured on independently instrumented production systems. We treat genuine cross-engine transfer — training on one real engine and evaluating on another — as the open question this paper motivates but does not answer, and identify it explicitly as future work (FW1, Section 6).

- **PostgreSQL**: $n = 644$ expert-validated samples (the same dataset used to train IAHA's Phase 2 classifier in [1]).
- **MySQL**: $n = 200$ samples, generated under the same schema for this study.
- **MongoDB**: $n = 200$ samples, generated under the same schema for this study.

We did not generate $n = 644$ samples for MySQL and MongoDB to match PostgreSQL, and we state the reason explicitly rather than leaving it implicit: the PostgreSQL dataset is not newly generated for this paper but is reused, unchanged, from [1], where it underwent independent development and informal validation over the course of that earlier study. Generating a matching $n = 644$ sample for MySQL and MongoDB would not reproduce that same provenance — it would simply be more synthetic data from the same generator, which would not address the underlying asymmetry (real prior validation history vs. newly generated data) and could give a misleading impression of equivalence between the three datasets. We instead treat the size difference as a variable to be explicitly controlled for, which is exactly the role of the size-matched analysis in Section 4.2, rather than a difference to be papered over by generating more same-generator data for the other two engines.

Table 1 Shared Feature Schema Across Engines

Feature	Description
`replication_lag_mb`	Replication lag (MB)
`transactions_per_sec`	Transaction throughput
`cpu_percent`	CPU utilisation (%)
`latency_ms`	Query/operation latency
`trans_rolled_back`	Rolled-back transaction rate
`deadlocks`	Deadlock count
`hour_of_day`	Time-of-day indicator
`memory_usage_pct`	Memory utilisation (%)
`connections_active`	Active connection count
`wal_buffers_full`	Write-ahead/journal buffer saturation

Engine-agnostic feature design. Features were deliberately chosen as generic operational metrics rather than PostgreSQL-specific internals, to enable this cross-engine comparison without re-engineering the feature set per database. We note that `wal_buffers_full` is measured as a generic write-ahead/journal buffer saturation proxy; for MongoDB this corresponds functionally to oplog buffer pressure rather than a literal PostgreSQL WAL buffer, and we report it under this generic name rather than renaming it per-engine, to preserve direct cross-engine comparability of the same underlying measurement axis. Similarly, `deadlocks` and `trans_rolled_back` are reported as generic concurrency-conflict and transaction-failure indicators rather than engine-native terminology.

Synthetic data and class imbalance. As in [1] (Limitation L2), all three datasets are synthetically generated under expert-informed thresholds rather than collected from production traces. Class distributions are imbalanced across all three engines, with CONSISTENCY accounting for 85.0% (MongoDB), 78.5% (MySQL), and 79.5%

(PostgreSQL) of samples, and AVAILABILITY represented by only $n = 1$, $n = 3$, and $n = 5$ samples respectively (Table 2).

Table 2 Class Distribution by Engine

Mode	MongoDB	MySQL	PostgreSQL
CONSISTENCY	170 (85.0%)	157 (78.5%)	512 (79.5%)
ECONOMIC	17 (8.5%)	22 (11.0%)	57 (8.9%)
PERFORMANCE	12 (6.0%)	18 (9.0%)	70 (10.9%)
AVAILABILITY	1 (0.5%)	3 (1.5%)	5 (0.8%)
Total (n)	200	200	644

3.3 Classification Protocol

For each engine, we train an independent RandomForest classifier (200 trees, balanced class weighting, `random_state=42` for both the classifier and the cross-validation fold assignment, fixed throughout all experiments in this paper for reproducibility) using repeated stratified 5-fold cross-validation (10 repeats, 50 fold-level accuracy estimates per engine), matching the cross-validation protocol used in [1].

3.4 Alternative Classifier Comparison

As justified at length in Section 2.4, we restrict empirical comparison in this paper to the static heuristic baseline (Section 3.5) and to alternative ML classifiers on the same task (this section), rather than to the configuration-tuning systems or HA orchestration tools discussed in Section 2.2 and Section 2.1, since none of those systems can be run on this four-class mode-selection task without a redesign that would make any comparison uninformative. RandomForest is used throughout this paper because it was the model selected in [1], not because it was benchmarked against alternatives for this specific four-class mode-selection task. To address this directly, we additionally train three alternative classifiers under the identical protocol (same features, same repeated 10×5-fold stratified CV, same `random_state=42`): XGBoost (200 trees; all other hyperparameters left at xgboost 3.4.1's library defaults, i.e. no additional tuning beyond matching RandomForest's tree count), a logistic regression (`class_weight="balanced"`, `max_iter=1000`, default `lbfgs` solver), and an RBF-kernel SVM (`class_weight="balanced"`, default kernel coefficient), both with balanced class weighting where supported and with features standardised (zero mean, unit variance) for the two distance/gradient-based models. All four classifiers use `random_state=42`, matching the seed used everywhere else in this paper. This does not compare our classifier against the ML-driven tuning systems discussed in Section 2.2 (OtterTune, GPTuner, and similar), since, as discussed in Section 2.4, those systems solve a different task (continuous parameter tuning) and cannot be run on a four-class mode-selection problem without redesign; it instead answers the narrower and directly answerable question of whether RandomForest is competitive with other standard classifiers on exactly this task.

3.5 Statistical Testing

BCa bootstrap confidence intervals. Following the methodology of [1], we report bias-corrected and accelerated (BCa) bootstrap 95% confidence intervals (10,000 resamples) on mean cross-validated accuracy per engine.

McNemar test. To test whether the RandomForest classifier significantly outperforms a static heuristic, we apply McNemar's test *within each engine*, comparing paired correct/incorrect predictions on the same held-out folds. The static heuristic mirrors the threshold logic of IAHA's P1 layer: `cpu_percent` $<$ 30 $\Rightarrow$ ECONOMIC; 30 $\leq$`cpu_percent` $<$ 50 $\Rightarrow$ PERFORMANCE; otherwise CONSISTENCY (the heuristic cannot express AVAILABILITY). We do *not* apply McNemar across engines (e.g. MongoDB vs. MySQL), since the two samples are drawn from different observation spaces and are not paired — this would violate the test's matched-pairs assumption.

Mann–Whitney U test. To compare the distributions of cross-validated accuracy between pairs of engines, we use the Mann–Whitney U test on the 50 fold-level accuracy estimates per engine. We note as a limitation that these 50 estimates per engine are not fully independent (they derive from repeated resampling of the same underlying dataset), so the resulting p-values likely overstate the true level of statistical confidence; we report

them as indicative rather than definitive evidence of inter-engine difference.

Multiple comparisons. The Mann–Whitney comparisons are grouped into three families of three tests each: (F1) the pairwise engine-accuracy comparisons (Section 4.1); (F2) the comparisons of size-matched PostgreSQL against full-size PostgreSQL, MySQL, and MongoDB (Section 4.2); and (F3) the pairwise total-RTO comparisons (Section 4.7). Within each family we report the unadjusted p-value, the Bonferroni-adjusted value ($p_{\text{Bonf}} = \min(1, 3p)$), and the Holm-adjusted value (p_{Holm}), which controls the same family-wise error rate with greater power. A difference is called significant if $p_{\text{Bonf}} < 0.05$; a value only marginally below that threshold (here $p_{\text{Bonf}} = 0.04998$) is called borderline and not treated as established, because it does not survive a different but equally defensible choice of family (see the sensitivity check below). Holm values are reported for completeness. Adjustment does not remove the non-independence of repeated-CV estimates (above) or the orchestration confound of the live comparison, so all three families are exploratory, and unadjusted p-values are reported as indicative evidence, distinct from corrected significance. The three McNemar tests (one per engine) are adjusted in the same way: the largest unadjusted value (2.46×10^{-7}) becomes 7.4×10^{-7} after Bonferroni correction, so all three remain below 10^{-6}. Adjusted values are computed from unrounded p-values and may differ in the last digit from values recomputed from the rounded figures printed in the text. As a sensitivity check, pooling all nine Mann–Whitney tests into a single family (Bonferroni factor 9) leaves exactly the same comparisons significant, except that the borderline F2 comparison with MySQL ($p_{\text{Bonf}} = 0.15$) is then clearly non-significant.

3.6 Sample-Size Control Protocol

Because PostgreSQL ($n = 644$) is 3.2× larger than MySQL or MongoDB ($n = 200$ each), any raw accuracy comparison between engines is confounded with a sample-size effect. To isolate the contribution of engine identity from dataset size, we additionally evaluate a *size-matched* PostgreSQL condition: 20 independent stratified subsamples of $n = 200$ are drawn from the full PostgreSQL dataset (approximately class-proportional: 158–159 CONSISTENCY, 21–22 PERFORMANCE, 17–18 ECONOMIC, 2 AVAILABILITY; 22 PERFORMANCE cases in 19 of the 20 subsamples and 21 in the other), each evaluated under the same repeated stratified cross-validation protocol as the full dataset (10 repeats × 5 folds), yielding $20 \times 50 = 1000$ fold-level accuracy estimates in total. This size-matched PostgreSQL condition is then compared against (i) PostgreSQL at full size, and (ii) MySQL and MongoDB at their native $n = 200$, using the same Mann–Whitney procedure described above.

3.7 Live Multi-Engine Cluster Validation

In addition to the offline classification evaluation above, we conducted live failover testing on containerised clusters for all three engines, to measure end-to-end recovery time rather than only classification accuracy. This directly extends the Phase 1 CloudNativePG validation of [1] to MySQL and MongoDB.

Cluster setup and an explicit orchestration asymmetry. PostgreSQL was deployed via CloudNativePG v1.22.0 (PostgreSQL 16.1, 3 instances), the same production-grade Kubernetes operator used in [1]. MySQL was deployed via the MySQL Operator's `InnoDBCluster` resource (3 instances, Group Replication, 1 router instance). MongoDB was deployed as a bare 3-replica `StatefulSet` running `mongod --replSet` directly (MongoDB Community Server 7.0.0), *without* a dedicated MongoDB Kubernetes operator. We state this asymmetry explicitly rather than letting it pass unremarked: PostgreSQL and MySQL benefit from operator-managed orchestration (automated leader election tooling, health probes, and reconciliation loops built on top of each engine's native replication), while MongoDB's failover in this setup relies solely on the database's own internal replica-set election with no additional Kubernetes-level orchestration. This is a threat to the fairness of the cross-engine RTO comparison (Section 4.7) and is discussed further in Threats to Validity (Section 5.2); it means the comparison below is better read as "CloudNativePG-managed PostgreSQL vs. Operator-managed MySQL vs. unmanaged MongoDB" rather than as a clean,

orchestration-neutral comparison of the three database engines themselves.

Experimental environment. All live cluster tests and offline classification experiments were run on a single host under Windows 11 with WSL2 (Ubuntu), using Docker Desktop with the WSL2 backend and `kind` (Kubernetes-in-Docker) to provision one local cluster per engine (`kind-iaha-postgres`, `kind-iaha-mysql`, `kind-iaha-mongodb`). Host specification: Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz (4 physical cores, 8 logical processors), 16 GB RAM; Docker Desktop version 28.3.2 (build 578ccf6); `kind` version 0.20.0. No other CPU-intensive process was intentionally run on the host during data collection, though WSL2's shared-kernel virtualisation means host-level scheduling noise cannot be fully ruled out as a contributor to the timing variability discussed in Section 4.7 and Section 5.2. The offline classification experiments (Sections 3.3–3.5) were run under Python 3.13.14 with the package versions pinned in `requirements-lock.txt` in the accompanying repository, rather than the minimum versions given in `requirements.txt`, so that the published numbers can be reproduced exactly rather than only approximately.

Failover protocol. For each test, we identify the current primary/leader pod, delete it via `kubectl delete pod`, and measure two sequential, non-overlapping intervals: (i) `promotion_time_s`, the time from pod deletion until a new primary is detected, and (ii) a post-promotion stabilisation interval until the cluster reports all replicas healthy/online again. We refer to the sum of these two intervals as *total RTO* (total_rto_s = promotion_time_s + stabilisation_s) throughout this paper. We flag this definition explicitly because our measurement scripts label the second interval alone as `rto_s`; read in isolation, that field understates true end-to-end recovery time, since it excludes the promotion-detection phase that necessarily precedes it. All RTO figures reported below use the summed, total quantity.

Sample sizes and exclusions. PostgreSQL: 30/30 attempted failover tests completed successfully. MySQL: 15 tests were attempted before testing was halted after test 15 failed to reach 3 ONLINE Group Replication members within the 180s timeout (recorded as a sentinel value in the raw log); this left $n = 14$ valid measurements. MongoDB: 30/30 tests returned a result, but 10 of the 30 produced a stabilisation time tightly clustered around 0.65–0.73s – implausibly fast and implausibly uniform compared to the remaining 20 tests (4.36–19.39s) – which we judge to most likely reflect a measurement artefact (e.g. a stale replica-set status read immediately after pod deletion, before Kubernetes had actually evicted the terminating pod) rather than genuine sub-second recovery; we therefore report MongoDB results on the remaining $n = 20$ tests and flag the excluded 10 as an unresolved measurement question rather than silently discarding them without explanation. We were unable to conclusively confirm the root cause of this artefact and note it as a limitation (Section 5.1).

4 Results

Results are reported in the order of the research questions: cross-engine classification performance (RQ1, Section 4.1), the sample-size control (RQ1b, Section 4.2), the alternative-classifier comparison (RQ1c, Section 4.3), feature-importance stability (RQ2, Section 4.4), a note on ECONOMIC separability (Section 4.5), the AVAILABILITY failure mode (RQ3, Section 4.6), and live recovery-time validation (RQ4, Section 4.7). Pairwise comparisons are interpreted within the families and the correction procedure defined in Section 3.5; the size-control, SMOTE, and live-failover analyses are exploratory.

4.1 RQ1: Cross-Engine Classification Performance

Table 3 summarises classification performance across the three engines.

All three classifiers substantially exceed both the majority-class baseline and the static heuristic. We note that on every engine the static heuristic scores below the majority-class baseline (MongoDB 79.00% vs. 85.00%; MySQL 75.00% vs. 78.50%; PostgreSQL 79.19% vs. 79.50%): the heuristic actively misclassifies some CONSISTENCY samples into ECONOMIC or PERFORMANCE when `cpu_percent` crosses its thresholds, whereas a baseline that always predicts CONSISTENCY incurs no such cost; this is consistent with the heuristic providing no information about

Table 3 Classification Performance by Engine (Repeated 10 × 5-Fold CV)

Metric	MongoDB	MySQL	PostgreSQL
Mean accuracy	97.55%	95.90%	98.62%
BCa 95% CI	[96.9, 98.1]	[95.3, 96.6]	[98.4, 98.8]
Majority baseline	85.00%	78.50%	79.50%
Heuristic acc.	79.00%	75.00%	79.19%
Macro-F1 (3 cl.)[1]	93.15%	93.83%	98.93%

[1]CONSISTENCY, ECONOMIC, PERFORMANCE only; AVAILABILITY excluded from aggregate macro-F1 due to insufficient sample size for reliable estimation (see Section 4.6). Computed by pooling predictions across all 50 repeated-CV folds (10 repeats × 5 folds) and taking a single macro-averaged F1 over the three classes on the pooled predictions, using scikit-learn's `f1_score(..., average="macro", zero_division=0)`; this matches `src/train_classifiers.py` in the accompanying repository exactly.

AVAILABILITY and a CPU threshold that only weakly separates CONSISTENCY from PERFORMANCE (Section 4.6). The heuristic is therefore a weak baseline, and the RandomForest gain over the majority-class baseline is smaller than over the heuristic (+12.6, +17.4 and +19.1 percentage points for MongoDB, MySQL and PostgreSQL). The RandomForest classifier does not exhibit this issue on any engine. McNemar's test confirms the RandomForest classifier significantly outperforms the static heuristic on every engine (Table 4). Note that the RF accuracy reported in Table 4 is computed on a single stratified 5-fold split (required for McNemar's paired-prediction design) rather than the repeated 10 × 5-fold protocol used for Table 3; the two are not directly comparable and the small numerical difference (e.g. 98.9% vs. 98.62% for PostgreSQL) reflects this difference in protocol, not an inconsistency in the underlying model.

Table 4 McNemar Test: RandomForest vs. Static Heuristic (Per Engine)

Engine	RF acc.	Heur. acc.	Δ (pp)	McNemar p
MongoDB	97.0%	79.0%	+18.0	2.5×10^{-7}
MySQL	96.0%	75.0%	+21.0	1.3×10^{-8}
PostgreSQL	98.9%	79.2%	+19.7	3.5×10^{-28}

The magnitude of improvement (+18.0 to +21.0 percentage points) is consistent with the +25.7 percentage-point improvement over static heuristics reported for PostgreSQL in [1], independently confirming on two additional engines that ML-driven mode selection provides a substantial, statistically significant advantage over fixed thresholds.

Per-class precision and recall. Table 3 and Table 4 report only aggregate accuracy and macro-F1; Table 5 completes the picture with per-class precision and recall for all four classes, computed on the same pooled predictions (all 50 repeated-CV folds concatenated) used for the pooled macro-F1 in Table 3, so the two are directly consistent. CONSISTENCY and ECONOMIC are both recovered with precision and recall at or above 95% on every engine (ECONOMIC reaching 100% throughout, consistent with its empirical separability discussed in Section 4.5); PERFORMANCE recall is lower and more variable across engines (95.3% PostgreSQL, 71.1% MySQL, 67.5% MongoDB), consistent with the training-set-size effect on this class discussed in Section 4.2; AVAILABILITY recall is 0% on every engine, discussed separately in Section 4.6 given the very small per-engine sample sizes involved.

Engine-level differences. A Mann–Whitney U test on fold-level accuracy distributions found all three pairwise comparisons (family F1) significant after Bonferroni correction (MongoDB vs. MySQL: $U = 1739.0$, $p = 3.6 \times 10^{-4}$, $p_{\text{Bonf}} = 1.1 \times 10^{-3}$; MongoDB vs. PostgreSQL: $U = 798.0$, $p = 0.0016$, $p_{\text{Bonf}} = 0.0048$; MySQL vs. PostgreSQL: $U = 240.0$, $p = 1.9 \times 10^{-12}$, $p_{\text{Bonf}} = 5.7 \times 10^{-12}$; Holm-adjusted values are 7.2×10^{-4}, 0.0016, and 5.7×10^{-12}), though as discussed in Section 3.5 these p-values should be read as indicative rather than definitive given the non-independence of repeated-CV estimates. The comparison families and the adjustment procedure are defined in Section 3.5; the corresponding adjustments for the size-control (F2) and total-RTO (F3) comparisons are reported in Sections 4.2 and 4.7, where some conclusions do not survive correction. PostgreSQL — the largest dataset ($n = 644$) — achieves the highest accuracy and the tightest confidence interval, which raises an immediate confound already flagged in Section 3: is PostgreSQL's apparent advantage a property of the engine, or simply a property of having 3.2× more training data? We test this directly in Section 4.2.

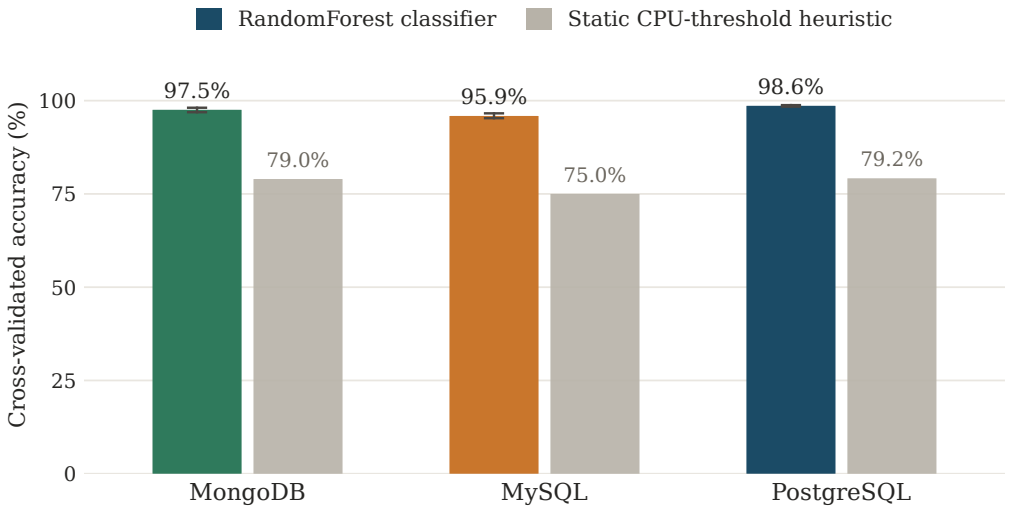


Fig. 1 RandomForest classifier accuracy versus the static CPU-threshold heuristic, by engine. Error bars show BCa 95% confidence intervals on the repeated 10 × 5-fold CV accuracy (Table 3); heuristic bars correspond to Table 4.

Table 5 Per-Class Precision and Recall (Pooled Predictions, All 50 Repeated-CV Folds)

	MongoDB		MySQL		PostgreSQL	
Class	Prec.	Rec.	Prec.	Rec.	Prec.	Rec.
CONSISTENCY	97.2%	100.0%	95.0%	100.0%	98.4%	99.9%
ECONOMIC	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
PERFORMANCE	100.0%	67.5%	100.0%	71.1%	99.1%	95.3%
AVAILABILITY	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%

4.2 RQ1b: Disentangling Engine Identity from Sample Size

To test whether PostgreSQL's higher accuracy in Table 3 reflects engine-specific structure or simply its larger sample size, we apply the size-matched control described in Section 3.6: PostgreSQL subsampled to $n = 200$ (approximately class-proportional, 20 subsamples, each evaluated under the same 10 × 5 repeated-CV protocol as the full dataset, yielding 1000 fold-level estimates in total) is compared against PostgreSQL at full size and against MySQL/MongoDB at their native $n = 200$.

The size effect itself is present but more modest than a first look at Table 3 might suggest: once PostgreSQL is reduced to the same sample size as MySQL and MongoDB, its accuracy drops by 1.93 percentage points (from 98.62% to 96.69%), and its confidence interval no longer overlaps with the full-size estimate (Mann–Whitney $U = 12556.0$, $p = 6.7 \times 10^{-10}$, $p_{\text{Bonf}} = 2.0 \times 10^{-9}$ within family F2). **This indicates that part of the performance advantage attributed to PostgreSQL in Section 4.1 is driven by dataset size, not by engine identity — though, as the comparison below shows, size is not the whole story.**

Table 6 Sample-Size Control: PostgreSQL Full vs. Size-Matched

Condition	Mean acc.	BCa 95% CI
PostgreSQL, full ($n = 644$)	98.62%	[98.40, 98.79]
PostgreSQL, matched ($n = 200$)	96.69%	[96.53, 96.86]
MySQL, native ($n = 200$)	95.90%	[95.30, 96.55]
MongoDB, native ($n = 200$)	97.55%	[96.90, 98.05]

The per-class breakdown (Table 7) localises part of this effect in the PERFORMANCE class (the sample-size-matched subsample also contains only $n = 2$ AVAILABILITY cases, too few to report a recall figure for that class here;

see Section 4.6 for the AVAILABILITY analysis): PERFORMANCE recall drops from 95.3% (full PostgreSQL) to 79.5% (size-matched PostgreSQL), a 15.8-point drop, once the dataset contains only 22 PERFORMANCE cases (about 17.6 per training fold) instead of 70 (about 56.0 per training fold). This is a real effect of reduced training data, but it does *not* bring size-matched PostgreSQL down into the same recall range as MySQL (71.1%) or MongoDB (67.5%); it remains above both (8.4 and 12.0 points, respectively). Sample size affects the PERFORMANCE class, but not enough on its own to equalise it with the other two engines at the same nominal training size. ECONOMIC remains at 100% regardless of sample size, consistent with its empirical separability discussed in Section 4.5.

Table 7 PERFORMANCE-Class Recall vs. Training Set Size. Cases: total PERFORMANCE examples in the dataset; Train/fold: mean per training fold; Recall: pooled over all repeated-CV test folds (size-matched: all 20 subsamples; 22 cases in 19, 21 in one).

Condition	Cases	Train/fold	Recall
PostgreSQL, full (n = 644)	70	56.0	95.3%
PostgreSQL, matched ($n = 200$)	22	17.6	79.5%
MySQL ($n = 200$)	18	14.4	71.1%
MongoDB ($n = 200$)	12	9.6	67.5%

Notably, the size-matched PostgreSQL condition does not fall below both other engines: it is nominally *higher* than MySQL native (95.90%; Mann–Whitney $U = 29797.0$, $p = 0.01666$, $p_{\text{Bonf}} = 0.04998$, $p_{\text{Holm}} = 0.033$) and nominally *lower* than MongoDB native (97.55%; $U = 20607.0$, $p = 0.0282$, $p_{\text{Bonf}} = 0.085$, $p_{\text{Holm}} = 0.033$), landing between the two in point estimate rather than below both. Within family F2 (Section 3.5), the MySQL comparison is borderline ($p_{\text{Bonf}} = 0.04998$, just under the 0.05 threshold, and not robust to pooling all tests into one family; Section 3.5) and the MongoDB comparison does not pass it, so we treat both as exploratory, unadjusted evidence rather than as established differences. Holm-adjusted values are reported for completeness; significance is judged by the more conservative Bonferroni criterion defined in Section 3.5. We interpret this pattern cautiously rather than as a settled finding. Having confirmed (Limitation L1) that all three datasets are produced by the same generator and the same random seed, a difference in generation procedure is not a plausible explanation. The more likely candidates are genuine finite-sample properties of how each engine's particular scenario mix shapes the four-class decision boundary at $n = 200$, or sampling variance in the 20-subsample protocol itself. We do not have grounds to attribute this ordering to a property of the PostgreSQL, MySQL, or MongoDB *engine* as such, and flag it as an open question for future work (Section 6, FW4 below) rather than a settled finding.

Implication for RQ1. The evidence supports a measured answer to RQ1: classifier performance is *not* fully consistent across engines at matched sample sizes, and training-set size is *part* of the explanation for the cross-engine differences observed in Table 3 — narrowing PostgreSQL's accuracy advantage over MySQL from 2.72 points (native sample sizes) to 0.79 points once matched (PostgreSQL's own accuracy drops by 1.93 points once reduced to $n = 200$), and clearly affecting PERFORMANCE-class recall — but size alone does not fully equalise the three engines, since size-matched PostgreSQL still lands between, rather than below, MySQL and MongoDB native performance in point estimate (an exploratory ordering that is borderline or non-significant after multiple-comparison correction). Within the controlled synthetic design tested here (Section 3.2), we conclude that both training-set size and some remaining engine- or dataset-specific factor contribute to the cross-engine differences, with size accounting for a substantial but not exhaustive share. The practical implication is correspondingly modest: the data collection budget per class likely matters for non-dominant modes such as PERFORMANCE, but it should not be assumed to fully substitute for engine-specific validation.

4.3 RQ1c: Is RandomForest the Right Classifier?

Every result so far uses RandomForest because it was the model selected in [1], not because it was benchmarked against alternatives for this specific task. Table 8 addresses this directly, comparing RandomForest against XGBoost, a multinomial

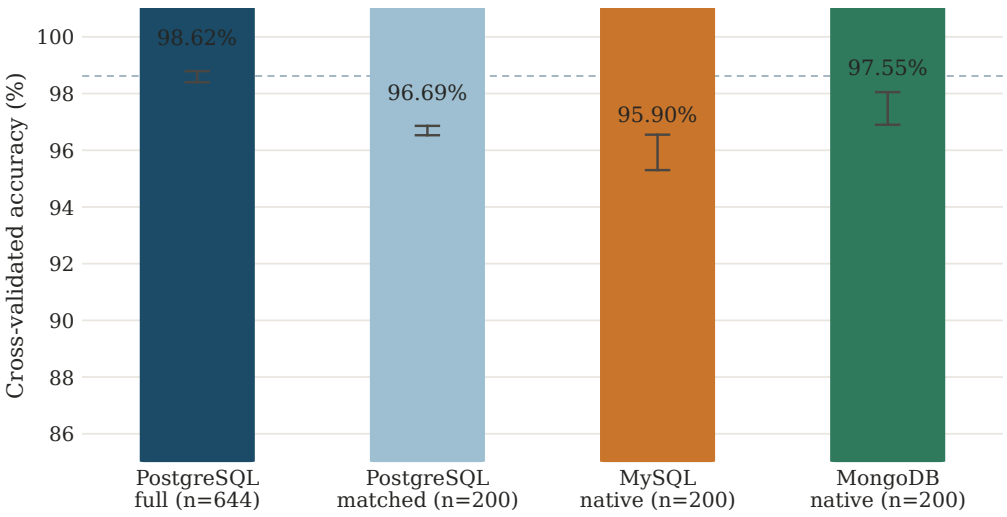


Fig. 2 Sample-size control: mean cross-validated accuracy for PostgreSQL at full size, PostgreSQL size-matched to $n = 200$, and MySQL/MongoDB at their native $n = 200$. Error bars show BCa 95% confidence intervals (Table 6); the dashed line marks full-size PostgreSQL accuracy for reference. The y-axis is truncated (lowest tick shown: 86%), so differences look larger than they are; the largest gap shown is 2.72 points.

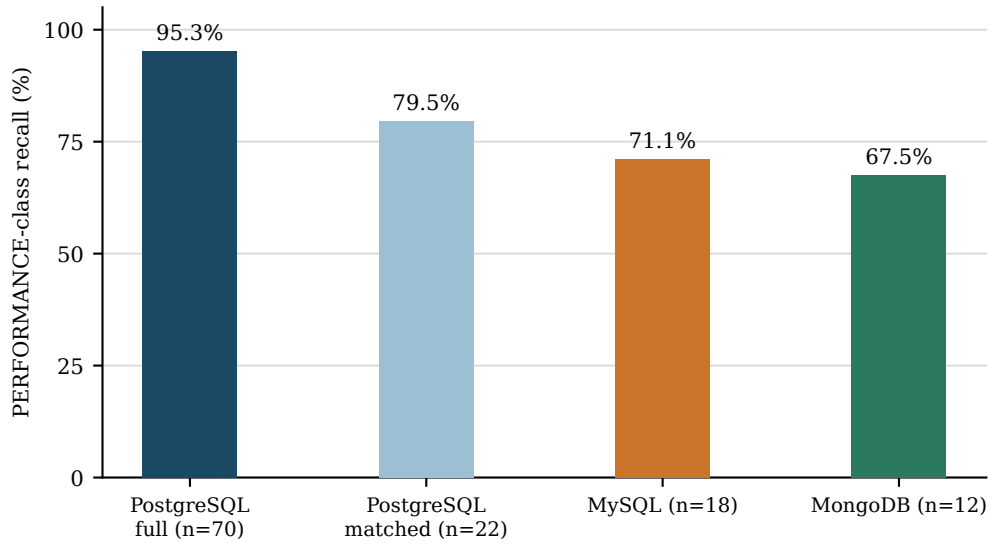


Fig. 3 PERFORMANCE-class recall (pooled over all repeated-CV test folds) for the four conditions in Table 7; n in the axis labels is the total number of PERFORMANCE cases in each dataset, and the mean number per training fold is given in that table.

logistic regression, and an RBF-kernel SVM under the identical protocol (Section 3.4).

Two patterns are clear and, unlike several other results in this paper, do not require heavy qualification. First, RandomForest is the best classifier, or within 0.5 percentage points of the best, on every engine: it has the highest accuracy on all three engines, and the highest macro-F1 on two of three (MongoDB, MySQL), with XGBoost marginally ahead on PostgreSQL macro-F1 by 0.04 percentage points — well within fold-to-fold noise. RandomForest was not, therefore, an arbitrary or merely inherited choice: it is empirically

Table 8 Classifier Comparison: Accuracy by Engine (Repeated 10 × 5-Fold CV)

Classifier	MongoDB	MySQL	PostgreSQL
RandomForest	**97.55%**	**95.90%**	**98.62%**
XGBoost	97.10%	95.35%	98.54%
LogisticRegression	84.20%	84.35%	86.06%
SVM (RBF)	83.65%	76.95%	77.16%

Table 9 Classifier Comparison: Pooled 3-Class Macro-F1 by Engine

Classifier	MongoDB	MySQL	PostgreSQL
RandomForest	**93.15%**	**93.83%**	98.93%
XGBoost	90.99%	92.92%	**98.97%**
LogisticRegression	77.46%	84.08%	85.47%
SVM (RBF)	75.20%	75.92%	77.86%

competitive with a stronger, more recent tree-ensemble alternative (XGBoost) on exactly this task. Second, and more strikingly, both tree-based methods (RandomForest, XGBoost) outperform both linear/kernel methods (logistic regression, SVM) by a large and consistent margin across all twelve tree-vs-linear/kernel pairings — from 11.0 percentage points (XGBoost vs. logistic regression, MySQL) to 21.5 percentage points (RandomForest vs. SVM, PostgreSQL) in accuracy. This is not a close contest. We read this as consistent with, though not proof of, the underlying data-generating process described in Limitation L1: labels are assigned by a deterministic rule cascade over threshold conditions on the feature values (Section 3.1), which produces decision boundaries that are naturally piecewise and axis-aligned — exactly the structure tree ensembles are suited to, and exactly what a global linear decision boundary (logistic regression) or a smooth kernel boundary (RBF-SVM) struggles to approximate. This does not mean tree-based methods would necessarily dominate on real, non-synthetic operational data with a different underlying structure, and we do not claim otherwise; it does mean that, on the specific task and data studied in this paper, the choice of RandomForest is empirically justified rather than a default carried over unexamined from prior work.

Table 10 Top-4 Feature Importances by Engine (RandomForest)

Feature	MongoDB	MySQL	PostgreSQL
`transactions_per_sec`	0.227	0.279	0.238
`cpu_percent`	0.204	0.194	0.227
`connections_active`	0.150	0.149	0.171
`replication_lag_mb`	0.122	0.135	0.158

4.4 RQ2: Cross-Engine Feature Importance Stability

The same four features dominate the decision process across all three engines, in the same relative order. Spearman rank correlation between full feature-importance rankings (all ten features) is $\rho = 0.806$ (MongoDB vs. MySQL), $\rho = 0.915$ (MongoDB vs. PostgreSQL), and $\rho = 0.891$ (MySQL vs. PostgreSQL). We emphasise what this correlation does and does not show: it indicates that the *relative ranking* of which features matter is stable across these three parametric variations of a single generator (Section 3.2), which is a necessary precondition for transfer learning to be worth attempting (there would be little reason to transfer a model if the engines relied on different signals entirely). It is not, by itself, evidence that a classifier's learned decision boundary transfers to independently instrumented engines — that would require directly training on one real engine and testing on another, which we identify as future work (FW1, Section 6) rather than claim here. We therefore describe the decision signature as *structurally consistent* under this controlled design, and treat genuine transferability as an open empirical question.

4.5 A Note on ECONOMIC Separability

Throughout this paper, the ECONOMIC class is perfectly separable (recall and precision = 100% on all three engines and at all sample sizes tested), and our static heuristic baseline reproduces this separation using a single threshold, `cpu_percent` < 30 (Section 3.5). It is important to be precise about *why* this threshold works, since the underlying data-generation process (consulted directly rather than inferred) reveals that `cpu_percent` is not itself the generating rule for

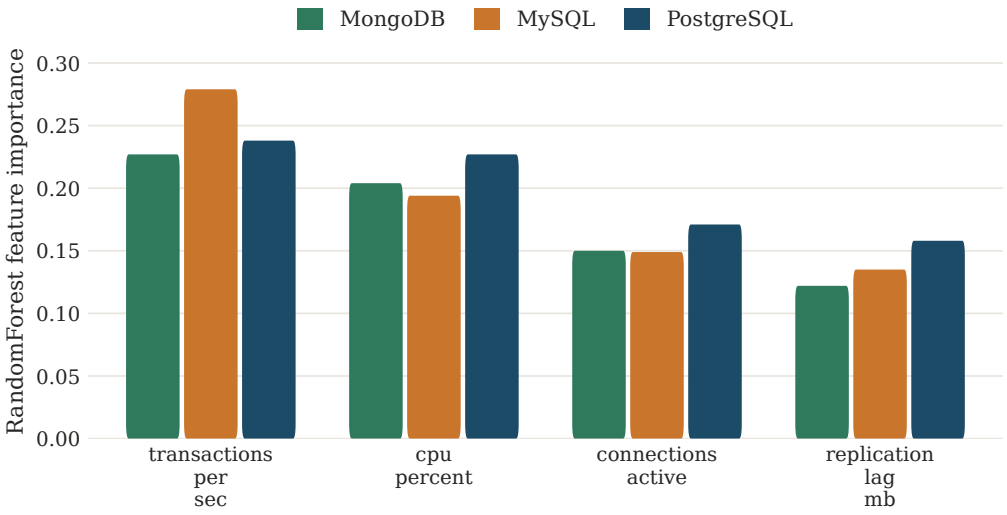


Fig. 4 Top-4 RandomForest feature importances by engine (Table 10). The same four features dominate in the same relative order on all three engines.

ECONOMIC. ECONOMIC samples are generated under a low-load "off-peak" scenario defined jointly by night-time hours and low transaction throughput; CPU utilisation in that scenario happens to be drawn from a low range (10–30%) as a *consequence* of the same scenario, not as its defining criterion. The `cpu_percent` < 30 threshold is therefore an effective *empirical proxy* for this scenario, not a description of the causal mechanism that produced the label. We flag this distinction explicitly because it affects how the perfect separability of ECONOMIC should be read: it demonstrates that a single operational metric can capture a multivariate, time-of-day-dependent operating regime well enough to be perfectly diagnostic on this dataset, which is a more interesting empirical finding than a merely definitional threshold would be, but it should not be over-interpreted as evidence that CPU utilisation alone *causes* or *defines* the ECONOMIC mode in a deployed system.

4.6 RQ3: The AVAILABILITY Failure Mode

Across all three engines, every AVAILABILITY misclassification goes to CONSISTENCY, never to ECONOMIC or PERFORMANCE (Table 11). This directional pattern is consistent across engines regardless of sample size and is the most robust part of this result. The corresponding recall figure is 0% in every engine, but we treat this number with caution rather than as a precise estimate: with only $n = 1$, 3, and 5 AVAILABILITY samples respectively, recall is not statistically distinguishable from what a single unlucky draw would produce, and we report it descriptively.

Table 11 AVAILABILITY Misclassification Pattern by Engine

Engine	n_{avail}	Pred. CONS.	Recall	Overlap[2]
MongoDB	1	1/1	0%	n/a[3]
MySQL	3	3/3	0%	33.2%
PostgreSQL	5	5/5	0%	38.9%

[2]Mean % of CONSISTENCY samples falling within the observed AVAILABILITY feature range, averaged across the ten features ("feature overlap").

[3]Not statistically interpretable with $n = 1$.

Pred. CONS.: number of AVAILABILITY test samples predicted as CONSISTENCY.

As a diagnostic, to probe whether this failure is a correctable sample-size artefact or reflects a structural overlap, we ran an oversampling experiment: SMOTE applied to the PostgreSQL training folds (the only engine with enough AVAILABILITY samples, $n = 5$, to make oversampling meaningful). Because stratified 5-fold CV with $n = 5$ AVAILABILITY samples places exactly one such sample in each held-out test fold,

every training fold contains exactly four AVAILABILITY samples; we therefore set SMOTE's neighbourhood-size parameter to `k_neighbors=3` (one below the number of available minority samples, the maximum value SMOTE permits) rather than its default of 5, which would not run with fewer than six minority samples per training fold. SMOTE improved AVAILABILITY recall from 0% to only 20% (F1 = 0.29), with precision dropping to 50%. This diagnostic oversampling experiment was not repeated on MySQL or MongoDB, as $n = 1$ and $n = 3$ AVAILABILITY samples are too few to support oversampling ($n = 3$ for MySQL would permit at most `k_neighbors=1` within a training fold containing only 2–3 such samples, which we judged too degenerate to be informative). Together with the descriptive feature-overlap figures (33–39% for MySQL and PostgreSQL; not statistically interpretable for MongoDB at $n = 1$), the limited gain is compatible with a structural overlap between AVAILABILITY and CONSISTENCY in the current ten-feature schema. It is equally compatible with other explanations, most notably the very small number of independent minority examples (five on PostgreSQL, from which SMOTE can only interpolate), and this experiment cannot separate the two. We therefore treat the structural-overlap reading strictly as a hypothesis for future work (FW2), not as a finding. The pattern is consistent with Limitation L3 of [1] ("CONSISTENCY = 89.4% limits generalisation for ECONOMIC and AVAILABILITY"), but given the sample sizes involved it does not show that this limitation is shared across engines or independent of sample size.

4.7 RQ4: Live Recovery-Time Validation Across Engines

Table 12 reports total RTO (Section 3.7) for all three engines under the live failover protocol, using the valid tests only (30 of 30 for PostgreSQL, 14 of 15 for MySQL, 20 of 30 for MongoDB; see Section 3.7 for the exclusions). Table 13 gives the corresponding pairwise tests.

Within family F3 (Table 13), two of the three pairwise Mann–Whitney comparisons remain significant after Bonferroni correction (PostgreSQL vs. MySQL: $U = 0.0$, $p = 1.3 \times 10^{-7}$, $p_{\text{Bonf}} = 3.9 \times 10^{-7}$, rank-biserial $r = 1.00$; MySQL vs. MongoDB: $U = 280.0$, $p = 1.1 \times 10^{-6}$, $p_{\text{Bonf}} = 3.2 \times 10^{-6}$, $|r| = 1.00$). The third, PostgreSQL vs. MongoDB ($U = 199.0$, $p = 0.047$, $r = 0.34$), does not: $p_{\text{Bonf}} = 0.14$ (Holm: 0.047), so we regard it as suggestive but not established. Given the orchestration asymmetry stated in Section 3.7, and because this family is exploratory, we read these as differences between *deployments* rather than differences intrinsic to the database engines themselves. The rank-biserial correlation ($r = 1 - 2U/(n_1 n_2)$) is large for two of the three comparisons: PostgreSQL vs. MySQL and MySQL vs. MongoDB both reach $|r| = 1.00$, meaning every test in the faster engine's sample recorded a lower total RTO than every test in the slower engine's sample (complete stochastic dominance, not merely a significant average difference). PostgreSQL vs. MongoDB is a more moderate effect ($r = 0.34$), consistent with their total RTO ranges (Table 12: 25.77–57.65 s and 35.67–52.33 s) overlapping, whereas MySQL's range (104.54–107.43 s) overlaps neither.

MySQL Group Replication is slowest but by far the most predictable. At 105.29s mean total RTO, MySQL GR is 2.4–2.8× slower than PostgreSQL or MongoDB in this setup, but its coefficient of variation (0.9%) is an order of magnitude tighter than either alternative. This is consistent with MySQL Group Replication's failure detection being dominated by fixed, configuration-driven timeout parameters (e.g. `group_replication_member_expel_timeout` and related settings) rather than by variable network- or consensus-driven election dynamics, making its recovery behaviour highly predictable even though slower in absolute terms.

PostgreSQL shows a temporal ordering effect that we do not fully explain. Examining `promotion_time_s` in test order reveals two clusters rather than random scatter: the first ten tests average 34.02s, while tests 11–30 average 13.69s (with one late outlier at test 23, 42.46s). The gap between PostgreSQL's mean (37.26 s) and median (29.43 s) in Table 12 is consistent with a right-skewed total-RTO distribution. The ordering pattern is consistent with the CloudNativePG control loop, DNS propagation, or some other aspect of cluster state becoming faster to converge after the first several pod replacements in a run, but our test harness did not instrument enough internal state to confirm the mechanism,

Table 12 Total RTO by Engine (Live Kubernetes Failover Tests). Total RTO = `promotion_time_s` + stabilisation interval (Section 3.7), in seconds. Valid/attempted: MySQL excludes failed test 15 (180 s timeout); MongoDB excludes 10 tests with implausibly short stabilisation times (0.65–0.73 s). The CI is the BCa 95% interval of the mean.

Engine	Valid/attempted	Mean	Median	Min–Max	BCa 95% CI	CV
PostgreSQL	30/30	37.26	29.43	25.77–57.65	[33.20, 42.20]	34.2%
MySQL GR	14/15	105.29	104.93	104.54–107.43	[104.96, 105.95]	0.9%
MongoDB RS	20/30	43.82	46.78	35.67–52.33	[41.06, 46.58]	14.8%

Table 13 Pairwise Total-RTO Comparisons (Family F3; Mann–Whitney U, two-sided). $r = 1 - 2U/(n_1 n_2)$ is the rank-biserial correlation for the first-listed engine versus the second (negative: the first engine tended to have the higher RTO). $p_{\text{Bonf}} = \min(1, 3p)$. Holm values are given for completeness; significance is judged by the Bonferroni criterion (Section 3.5).

Comparison	U	p	p_{Bonf}	p_{Holm}	r
PostgreSQL vs. MySQL	0.0	1.3×10^{-7}	3.9×10^{-7}	3.9×10^{-7}	+1.00
PostgreSQL vs. MongoDB	199.0	0.047	0.14	0.047	+0.34
MySQL vs. MongoDB	280.0	1.1×10^{-6}	3.2×10^{-6}	2.1×10^{-6}	−1.00

and the 30 tests should therefore not be treated as independent, identically distributed draws from a single stationary RTO distribution — a point we return to in Threats to Validity (Section 5.2).

MongoDB's excluded fast cluster remains only partially explained. As described in Section 3.7, 10 of 30 MongoDB tests produced stabilisation times of 0.65–0.73s, tightly clustered and implausibly fast relative to the remaining 20 tests (4.36–19.39s). We exclude these 10 from Table 12 rather than either silently dropping them (as the original test script did for a different, unrelated reason) or including them as genuine sub-second recoveries without comment. We were not able to confirm the exact mechanism from the logs available to us; the most plausible explanation, given that MongoDB in this setup runs without a dedicated orchestration operator (Section 3.7), is a stale replica-set status read immediately following pod deletion, before Kubernetes had fully evicted the terminating pod from the service's endpoint list. We report this openly as an unresolved measurement question rather than asserting a specific cause we cannot verify.

5 Discussion

RQ1. One result here is unambiguous and needs no hedging: ML-driven mode selection significantly outperforms the static CPU-threshold heuristic on every engine, by 18–21 percentage points, at $p < 10^{-6}$ (Section 4.1). Whatever else this paper leaves open, that finding stands on its own. The more nuanced question is *why* accuracy varies across engines at all. Classification accuracy is high across all three engines but is not uniform, and our size-matched control (Section 4.2) indicates *empirically* that *part* of this non-uniformity is driven by training-set size, within the controlled synthetic design tested here: subsampling PostgreSQL to MySQL/MongoDB's native size narrows its accuracy advantage over MySQL from 2.72 to 0.79 points, drops PostgreSQL's own accuracy by 1.93 points, and noticeably reduces PERFORMANCE-class recall (from 95.3% to 79.5%). This is a useful, if more modest than initially expected, signal for practitioners — the data collection budget per class (especially for non-dominant modes such as PERFORMANCE and AVAILABILITY) plausibly matters, though evidently not enough on its own to equalise engines at matched sample size — and confirming this on independently instrumented production engines remains to be done (Section 3.2). Size does *not* fully explain the cross-engine differences: size-matched PostgreSQL nominally lands between, not below, MySQL and MongoDB accuracy (an exploratory ordering, borderline or non-significant after correction; Section 4.2), leaving a possible residual, dataset- or engine-specific effect that this study is not positioned to fully explain.

RQ2. The strong correlation in feature importance across the three engine variants ($\rho \geq 0.80$) suggests that `transactions_per_sec`, `cpu_percent`, `connections_active`, and `replication_lag_mb` constitute a largely engine-agnostic "signature" of replication-mode state *under this controlled design*. This is a necessary (though not sufficient) condition for transfer learning across engines: it justifies attempting a transfer-learning study — pre-training on PostgreSQL's larger dataset as a possible initialisation for MySQL or MongoDB classifiers, particularly for the data-scarce PERFORMANCE and AVAILABILITY classes — but does not itself demonstrate that such transfer would succeed on independently instrumented engines.

RQ3. The fact that AVAILABILITY is *never* confused with ECONOMIC or PERFORMANCE — only ever with CONSISTENCY — across all three engines is itself informative: it suggests the current feature schema captures a single dominant axis of variation (roughly, load/CPU intensity) that separates ECONOMIC and PERFORMANCE well, but does not capture whatever distinguishes an "available-but-degraded" state from a normal CONSISTENCY state. Candidate additional features for future work include active replica/read-replica counts, health-check failure rates, or failover-readiness indicators — none of which are present in the current ten-feature schema.

5.1 Limitations

L1 – Synthetic data, single shared generator. As introduced in Section 3.1 and discussed at length in Section 3.2, all three datasets are synthetically generated by a single shared Python generator rather than collected from live production traces, so this study should be read as a controlled synthetic evaluation rather than an empirical survey of real production systems (see also Threats to Validity, External). We give the remaining mechanical detail here for reproducibility. Each sample is drawn from one of five latent operating scenarios (normal, peak, off-peak, crisis, degraded), each with its own probability and its own parameter ranges for the ten operational features; the four-class `mode` label is then assigned by a deterministic rule cascade over the sampled feature values (load thresholds, error rates, deadlock counts, and time-of-day), not by a separate validation step applied to each generated row. "Expert-validated" in this context and in [1] refers to the scenario probabilities, parameter ranges, and labelling-rule cascade themselves having been specified by domain expertise, not to an independent post-hoc review of each individual sample. Two of the ten features (`wal_buffers_full`, `trans_rolled_back`) are drawn from engine-specific ranges within the shared generator (e.g. `wal_buffers_full` $\sim \mathcal{U}(0, 50)$ for PostgreSQL vs. $\mathcal{U}(0, 20)$ for MongoDB), consistent with the 2–2.5× scale divergence we measured empirically between engines on exactly these two features; all remaining features share identical generating ranges across engines.

One consequence of this design is worth drawing out explicitly for Section 4.2: because the same code and seed produce all three datasets, a discrepancy *between generators* cannot explain the residual, unattributed ordering identified there (size-matched PostgreSQL landing between MySQL and MongoDB at equal n, rather than below both). This narrows, but does not resolve, the candidate explanations for that residual; we discuss the remaining candidates — sampling variance and finite-sample structure specific to each engine's scenario mix — as an open question in FW4 (Section 6) rather than a settled finding.

L2 – Extreme AVAILABILITY imbalance. With $n = 1$, 3, and 5 AVAILABILITY samples respectively, no engine permits a statistically reliable estimate of AVAILABILITY recall; we report it descriptively rather than within aggregate metrics.

L3 – Non-independence of repeated-CV estimates. Mann–Whitney comparisons between engines use 50 fold-level accuracy estimates per engine drawn from repeated resampling of the same underlying dataset; these are not fully independent observations, so reported p-values likely overstate true statistical confidence. The pairwise comparisons are grouped into three exploratory families with Bonferroni and Holm adjustment (Section 3.5); adjustment does not remove this non-independence.

L4 – Live validation performed, but under an orchestration asymmetry and with unresolved measurement questions.

We extend the live Kubernetes validation of [1] (Phase 1, CloudNativePG) to MySQL and MongoDB (Section 4.7), so this study is no longer limited to offline classification performance. However, the live comparison is confounded by an orchestration asymmetry (PostgreSQL and MySQL run under production-grade operators; MongoDB does not, Section 3.7), by a temporal ordering effect in the PostgreSQL failover tests that we do not fully explain, and by an unresolved measurement artefact affecting 10 of 30 MongoDB tests. The live RTO results should therefore be read as an initial, imperfect validation rather than a definitive cross-engine comparison.

5.2 Threats to Validity

Internal. The static heuristic baseline reuses the threshold logic of IAHA's P1 layer rather than an independently tuned baseline per engine, which may understate what a carefully engine-tuned heuristic could achieve. Separately, the live PostgreSQL failover tests (Section 4.7) show a temporal ordering effect (early tests substantially slower than later ones) that we cannot fully attribute to a specific mechanism; the 30 tests should not be treated as i.i.d. draws from a single stationary distribution, which weakens the precision (though not necessarily the direction) of the PostgreSQL RTO estimate.

Construct. Feature semantics (e.g. `wal_buffers_full` as an oplog-buffer proxy for MongoDB) are functional analogues rather than native engine metrics; this trades engine-specific fidelity for cross-engine comparability, an explicit design choice discussed in Section 3. Separately, the live RTO comparison (Section 4.7) is confounded by an orchestration asymmetry: PostgreSQL and MySQL are managed by production-grade Kubernetes operators (CloudNativePG, MySQL Operator), while MongoDB runs as a bare StatefulSet with no dedicated operator. The reported RTO differences therefore reflect *deployment* differences at least as much as they reflect properties of the database engines themselves, and should not be read as a claim that MongoDB's replication mechanism is inherently slower or less reliable than PostgreSQL's or MySQL's.

External. As emphasised in Section 3.2, results are obtained on synthetic datasets of differing size (200/200/644) produced by a single shared generator, not on independently instrumented production traces; generalisation to production traces, to genuinely heterogeneous engine instrumentation, and to balanced multi-engine datasets of equal size all remain to be validated (cf. L1, L2). This is the most significant threat to the external validity of the paper's headline claims and should weigh heavily on how the results are interpreted.

6 Conclusion and Future Work

This study directly addresses FW4 from [1] by evaluating, in a controlled synthetic setting sharing a single generative process across engines (Section 3.1), whether ML-driven replication mode selection behaves consistently across PostgreSQL, MySQL, and MongoDB — a necessary precondition for, though not a demonstration of, cross-engine transfer (Section 3.2). We find that classification accuracy is consistently high and significantly outperforms static heuristics on all three engines (McNemar $p < 10^{-6}$), and that the underlying feature-importance signature is highly stable across the three engine variants (Spearman $\rho \geq 0.80$) — evidence that supports, but does not yet demonstrate, cross-engine transfer learning. A size-matched control experiment shows that the cross-engine accuracy differences observed at native dataset sizes are *partly* — but not fully — driven by training-set size rather than by intrinsic properties of the database engine, within this controlled design: reducing PostgreSQL to $n = 200$ narrows its accuracy advantage over MySQL from 2.72 to 0.79 points (PostgreSQL's own accuracy drops by 1.93 points) and noticeably affects PERFORMANCE-class recall, but size-matched PostgreSQL nominally still lands between, not below, MySQL and MongoDB, so a possible residual difference between engines remains unexplained (Section 4.2, FW4 below) and still requires validation on independently instrumented engines before either the size effect or the residual can be treated as an operational recommendation. At the same time, AVAILABILITY samples are consistently misclassified as CONSISTENCY, never as ECONOMIC or PERFORMANCE, on all

three engines; because per-engine AVAILABILITY counts are very small ($n = 1, 3, 5$), we treat this as a descriptive, hypothesis-generating pattern rather than a settled recall estimate, and the diagnostic oversampling experiment available on PostgreSQL is compatible with — but cannot establish — a gap in the current feature schema, alongside the equally plausible explanation of too few independent minority examples. Finally, the live failover comparison is exploratory and confounded by an orchestration asymmetry across engines, so its recovery-time differences should be read as differences between deployments rather than between database engines.

FW1. Explicit transfer-learning evaluation on independently instrumented engines (pre-train on PostgreSQL, fine-tune on real MySQL/MongoDB deployments), directly testing whether the feature-importance stability found here under a shared generator translates into sample-efficiency gains for the data-scarce engines in practice.

FW2. Extend the feature schema with availability-specific signals (replica health-check failure rate, active read-replica count) to test whether the hypothesised structural AVAILABILITY/CONSISTENCY overlap suggested in Section 4.6 can be resolved.

FW3. Resolve the two open measurement questions from the live validation in Section 4.7: confirm the mechanism behind the excluded fast MongoDB cluster (most plausibly a stale status read during pod eviction), and re-run PostgreSQL failover tests with sufficient repetitions and instrumentation to determine why early tests are consistently slower than later ones. Repeat the live validation with MongoDB under a dedicated Kubernetes operator to remove the orchestration asymmetry noted in Section 3.7, so that engine differences can be separated from deployment differences. Extend the preliminary RPO/data-durability piloting we have begun on MongoDB to PostgreSQL and MySQL under the same protocol.

FW4. Investigate the residual, unattributed ordering between size-matched PostgreSQL, MySQL, and MongoDB identified in Section 4.2 (size-matched PostgreSQL lands nominally above MySQL but nominally below MongoDB at the same $n = 200$; both comparisons are exploratory and borderline or non-significant after multiple-comparison correction). Having confirmed (Limitation L1) that all three datasets share a single generator and seed, a generator discrepancy is not a candidate explanation; remaining hypotheses are sampling variance not fully captured by our 20-subsample protocol, or genuine finite-sample structure in how the four-class decision boundary behaves under each engine's particular scenario mix at $n = 200$ specifically.

Acknowledgments. Not applicable.

Statements and Declarations

- **Funding:** The authors did not receive support from any organization for the submitted work.
- **Competing Interests:** The authors have no relevant financial or non-financial interests to disclose.
- **Ethics approval:** Not applicable.
- **Consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Availability of data and materials:** The datasets generated and analysed in this study, together with the data generation script, are publicly available in the GitHub repository https://github.com/imanejerrari-etu-bit/cross-db-replication-mode-selection under a CC-BY-4.0 license.
- **Code availability:** The analysis code used to produce the results in this paper, together with the dataset generation script, is available in the same repository (https://github.com/imanejerrari-etu-bit/cross-db-replication-mode-selection) under an MIT license.
- **Authors' contributions:** All authors contributed to the study conception and design. Material preparation, data collection, and analysis were performed by I. Jerrari. The first draft of the manuscript was written by I. Jerrari and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.
- **Use of AI tools:** A Large Language Model (LLM) tool assisted with manuscript preparation, including copy editing and reviewing the scope and framing of certain claims. All experiments, analyses, results, and interpretations are the authors' own, who take full responsibility for the manuscript's content. No generative AI tool is listed as an author.

References

[1] Jerrari, I., Assayad, I.: IAHA: An intelligent adaptive high availability framework for containerised PostgreSQL databases using hybrid machine learning. Journal of Intelligent & Fuzzy Systems, under review (2026)

[2] Patroni Project: Patroni: A template for PostgreSQL HA. https://github.com/patroni/patroni. Accessed Apr 2025

[3] repmgr Project: repmgr: PostgreSQL replication manager. https://repmgr.org/. Accessed Apr 2025

[4] Sorint.lab: Stolon: PostgreSQL cloud-native HA. https://github.com/sorintlab/stolon. Accessed Apr 2025

[5] EnterpriseDB/CNCF: CloudNativePG: Cloud-native PostgreSQL operator. https://cloudnative-pg.io/. Accessed Apr 2025

[6] Kraska, T., et al.: SageDB: A learned database system. In: Proc. CIDR (2019)

[7] Marcus, R., Negi, P., Mao, H., Zhang, C., Alizadeh, A., Kraska, T., Papaemmanouil, O., Tatbul, N.: Neo: A learned query optimizer. Proc. VLDB Endow. **12**(11), 1705–1718 (2019)

[8] Marcus, R., et al.: Bao: Making learned query optimization practical. In: Proc. ACM SIGMOD, pp. 1275–1288 (2021)

[9] Van Aken, D., Pavlo, A., Gordon, G.J., Zhang, B.: Automatic database management system tuning through large-scale machine learning. In: Proc. ACM SIGMOD, pp. 1009–1024 (2017). https://doi.org/10.1145/3035918.3064029

[10] Zhang, J., et al.: An end-to-end automatic cloud database tuning system using deep reinforcement learning. In: Proc. ACM SIGMOD, pp. 415–432 (2019)

[11] Rzadca, K., Findeisen, P., Swiderski, J., Zych, P., Broniek, P., Kusmierek, J., Nowak, P., Strack, B., Witusowski, P., Hand, S., Wilkes, J.: Autopilot: Workload autoscaling at Google. In: Proc. 15th Eur. Conf. Comput. Syst. (EuroSys '20), Heraklion, Greece, Article no. 16 (2020). https://doi.org/10.1145/3342195.3387524

[12] Qiu, H., et al.: FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices. In: Proc. OSDI, pp. 805–825 (2020)

[13] Gunning, D., et al.: XAI—Explainable artificial intelligence. Sci. Robot. **4**(37), eaay7120 (2019)

[14] Lundberg, S.M., Lee, S.-I.: A unified approach to interpreting model predictions. In: Proc. NeurIPS, pp. 4765–4774 (2017)

[15] Ribeiro, M.T., Singh, S., Guestrin, C.: "Why should I trust you?": Explaining the predictions of any classifier. In: Proc. ACM KDD, pp. 1135–1144 (2016)

[16] Wachter, S., Mittelstadt, B., Russell, C.: Counterfactual explanations without opening the black box: Automated decisions and the GDPR. Harvard J. Law Technol. **31**(2), 841–887 (2017)

[17] Guidotti, R., et al.: A survey of methods for explaining black box models. ACM Comput. Surv. **51**(5), 93:1–93:42 (2018). https://doi.org/10.1145/3236009

[18] Holzinger, A., Langs, G., Denk, H., Zatloukal, K., Müller, H.: Causability and explainability of artificial intelligence in medicine. WIREs Data Min. Knowl. Discov. **9**(4), e1312 (2019). https://doi.org/10.1002/widm.1312

[19] Atakishiyev, S., Salameh, M., Yao, H., Goebel, R.: Explainable artificial intelligence for autonomous driving: a comprehensive overview and field guide for future research directions. IEEE Access **12**, 101603–101625 (2024). https://doi.org/10.1109/ACCESS.2024.3431437

[20] Bussmann, N., et al.: Explainable machine learning in credit risk management. Comput. Econ. **57**, 203–216 (2021)

[21] Kanellis, K., Ding, C., Kroth, B., Müller, A., Curino, C., Venkataraman, S.: LlamaTune: Sample-efficient DBMS configuration tuning. Proc. VLDB Endow. **15**(11), 2953–2965 (2022). https://doi.org/10.14778/3551793.3551844

[22] Lao, J., Wang, Y., Li, Y., Wang, J., Zhang, Y., Cheng, Z., Chen, W., Tang, M., Wang, J.: GPTuner: A manual-reading database tuning system via GPT-guided Bayesian optimization. Proc. VLDB Endow. **17**(8), 1939–1952 (2024). https://doi.org/10.14778/3659437.3659449

[23] Wang, J., Trummer, I., Basu, D.: UDO: Universal database optimization using reinforcement learning. Proc. VLDB Endow. **14**(13), 3402–3414 (2021)